

① در تابع main با متغیر i تعریف شده و حلقه while با شرط $i < 3$ کار می کند. یعنی مادامی که $i < 3$ می باشد حلقه پایان نیابد. درون while از سوییچ کس (switch case) استفاده شده است که در آن مقادیر مختلف i از ۰ تا ۲ بررسی شدند که در هر کدام با برابری مقدار i با کس مورد نظر جدا ای چاپ می شود و تغییر i در مقدار i صورت می گیرد و با دستور break در پایان آن از switch خارج شده و شرط حلقه مجدداً بررسی می شود. در ضمن برای مقادیر دیگر i به جز بازه [۰ تا ۲] نیز در بخش default این دستورات صورت می پذیرند.

1. $\Rightarrow$ This is case 0

حال انجام دستورات که را بررسی می کنیم.
در ابتدا مقدار $i = 0$ است پس Case 0 برقرار است. با دستور $\text{Printf}(" \Rightarrow \text{This is case 0} \backslash n");$ مقدار i به صورت زیر در اکسیت می شود:
 $i = i + 2 = 0 + 2 = 2$
پس i آن چاپ شده و با دستور $i = i + 2$ مقدار i به صورت زیر در اکسیت می شود:
و پس با دستور break سوییچ بسته شده و شرط حلقه دوباره بررسی می شود؛ $i = 2$ پس Case 2 اجرا می شود:

2. $\Rightarrow$ This is case 2

ابتدا با دستور تابع $\text{Printf}(" \Rightarrow \text{This is case 2} \backslash n");$ عبارت رو به رو چاپ می شود و از مقدار i واحد کم می شود پس $i = i - 1 = 2 - 1 = 1$
می شود و پس دستور $i--$ / اجرا می شود و از مقدار i واحد کم می شود؛ حالا $i = 1$ پس Case 1 اجرا می شود با دستور break مجدداً سوییچ بسته می شود و شرط حلقه بررسی می شود؛

3. $\Rightarrow$ This is case 1

مطابق قبل عبارت رو به رو با دستور تابع Printf چاپ می شود و پس دستور $i = i + 2 = 1 + 2 = 3$ ؛ حال با دستور $i = i + 2$ اجرا می شود یعنی مقدار i اینگونه بزرگتر می شود: $i = 3$ است
break مجدداً سوییچ بسته می شود و به حلقه بر می گردد. حال شرط حلقه بررسی می شود و $i < 3$ است پس از حلقه خارج می شود و در نهایت دستور $\text{Printf}(" \backslash n \text{Program finished successfully} \backslash n");$ اجرا می شود.

خروجی کس که به صورت زیر است:

$\Rightarrow$ This is case 0

$\Rightarrow$ This is case 2

$\Rightarrow$ This is case 1

Program finished successfully!

#include <stdio.h>

(2)

```
int main() { int a=10, b=20, c=30
```

```
if (a>5) {
```

```
    if (b>25) {
```

```
        for (int i=0; i<1; i++) {
```

```
            printf("b is big\n");
```

```
            printf("a is small\n"); }
```

```
    else {
```

```
        printf("a is small\n"); } }
```

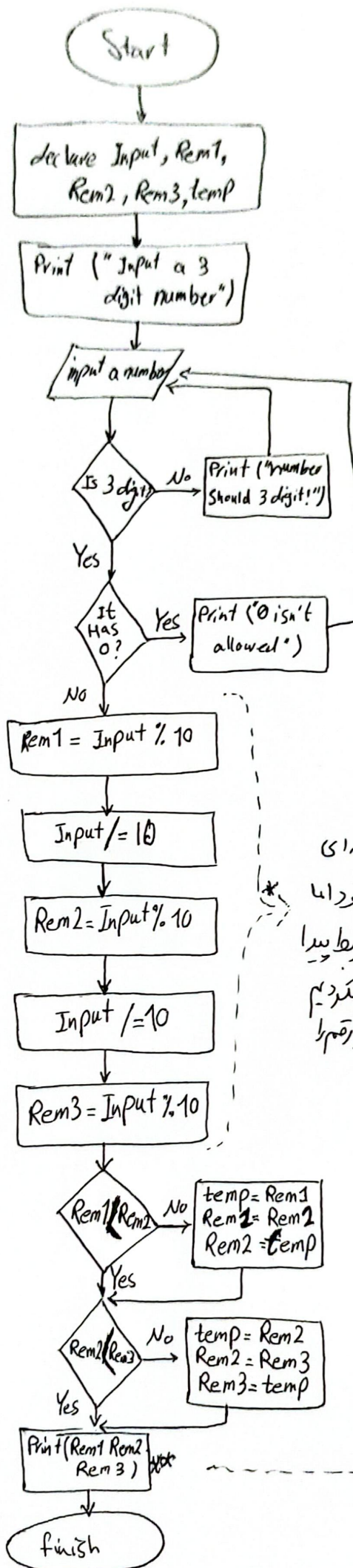
```
    for (double d=0.1; d<=2.0; d+=0.1)
```

```
    { printf("%f\n", d); }
```

```
}
```

```
a is small
0.10
0.20
0.30
0.40
0.50
0.60
0.70
0.80
0.90
1.00
1.1
1.2
1.3
1.4
1.5
1.6
1.7
1.8
1.9
```

خروجی :



* می توانیم این بخش را با حلقه ای نیز انجام دهیم تا کار خلاص تر شود اما حسن کردم به مبحث آرایه ها ربط پیدا می کند که هنوز کامل مطالعه نکردیم پس به صورت خط به خط هر رقم را جدا بررسی کردیم!

in(C)
Print("%d %d %d",
Rem1, Rem2, Rem3);

(E)

1. Start;
2. declare variable : Number, ~~Product~~ Product=1, i, Result.
3. Print ("Please enter a number");
4. input (Scan) Number;
5. ^{Start} for (i=Number; i >= 2; i--).
6. Product *= i; /* (Product = Product x i) */
7. End for;
8. Result = log (Product);
9. Print ("The Result is: Result") ~ in C: printf("The Result is: %d", Result);
10. Finish

(۵) ابتدا توابع فاکتوریل و توان را آماده می‌کنیم

Factorial Function: Factorial(n)

- 1- Start
- 2- Declare ~~For~~ variables: n, f, i ($f=1$)
- 3- ~~Set~~ For ($i=1; i \leq n; i++$):
- 4- $f = f \times i$ end For
- 5- Return f (in Factorial(n))

Power Function: Pow(x, m)

- 1- Start
- 2- Declare variables: $x, m, i, Result=1$
- 3- Start For ($i=1; i \leq m; i++$)
- 4- $Result *= x$ ($Result = Result \times x$)
- 5- End For.
- 6- Return Result.

حالا تابع اصلی سر در نظر بگیریم

- 1- Start
- 2- ~~declare~~ Variables: $n, x, k, Sign, Sum=0$
- 3- Print ("Enter n, x")
- 4- input n, x .
- 5- Start For ($k=0; k \leq n; k++$)
- 6- If ($k \% 2 == 0$) $\rightarrow$ $Sign=1$ End IF
- 7- ~~else~~ If ($k \% 2 == 1$) $\rightarrow$ $Sign=-1$ End IF
- 8- $Sum += Sign * (Pow(x, k) / Factorial(k))$
- 9- End For
- 10- Print ("The Result is: Sum")
- 11- End

- 1- Start
- 2- Declare variables: Number, i, rem[]
- 3- Print("Enter a decimal number")
- 4- input(scan) Number
- 5- while (Number != 0)
 - 6- ~~rem~~ rem[i] = Number % 2;
 - 7- number /= 2 ;
 - 8- i++;
 - 9- End while.
- 10- Print (reverse(rem[i]))
- 11- End

```
#include <stdio.h>
```

```
int main() {
    unsigned int number;
```

```
    int rem[32];
    int i=0;
    // 32 bit
```

```
    printf("Enter a decimal number")
```

```
    scanf("%u", &number)
```

```
    if (number == 0) {
        printf("0\n");
    }
```

```
    while (number != 0) {
        rem[i] = number % 2;
        number /= 2;
```

```
        i++;
```

```
    printf("The binary form of %d is: ", number);
```

```
    for (int j=i-1; j>=0; j--) { printf("%d", rem[j]); }
    return 0; }
```